

Application Security Verification Standard
(Level 1)

VIGIL-X

	Name	Date	Version
Created	Sai Pavan	24-06-2025	1.0
Reviewed	Gauri Shankar	01-07-2025	
Submission			

Table of Contents

V1 Encoding and Sanitization.....	4
V2 Validation and Business Logic.....	6
V3 Web Frontend Security.....	7
V4: API and Web Service Security	8
V5 File Handling.....	9
V6 Authentication.....	9
V7 Session Management.....	11
V8 Authorization.....	12
V9 Self-contained Tokens.....	13
V10 OAuth and OIDC	14
V11 Cryptography	14
V12 Secure Communication	15
V13 Configuration	16
V14 Data Protection	16
V15 Secure Coding and Architecture.....	17
References	17

V1 Encoding and Sanitization

(Refer to pages 23–28 in the linked document)

V1.2 Injection Prevention

ID	Description	Level	Reference
1.2.1	Verify that output encoding for an HTTP response, HTML document, or XML document is relevant for the context required, such as encoding the relevant characters for HTML elements, HTML attributes, HTML comments, CSS, or HTTP header fields, to avoid changing the message or document structure.	1	Link 1 Link 2
1.2.2	Verify that when dynamically building URLs, untrusted data is encoded according to its context (e.g., URL encoding or base64url encoding for query or path parameters). Ensure that only safe URL protocols are permitted (e.g., disallow javascript: or data:).	1	
1.2.3	Verify that output encoding or escaping is used when dynamically building JavaScript content (including JSON), to avoid changing the message or document structure (to avoid JavaScript and JSON injection).	1	
1.2.4	Verify that data selection or database queries (e.g., SQL, HQL, NoSQL, Cypher) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from SQL Injection and other database injection attacks. This is also relevant when writing stored procedures.	1	
1.2.5	Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding.	1	

V1.3 Sanitization

ID	Description	Level	Reference
1.3.1	Verify that all untrusted HTML input from WYSIWYG editors or similar is sanitized using a well-known and secure HTML sanitization library or framework feature.	1	Link
1.3.2	Verify that the application avoids the use of eval () or other dynamic code execution features such as Spring Expression Language (SpEL). Where there is no alternative, any user input being included must be sanitized before being executed.	1	

V1.4 Safe Deserialization

ID	Description	Level	Reference
1.5.1	Verify that the application configures XML parsers to use a restrictive configuration and that unsafe features such as resolving external entities are disabled to prevent XML eXternal Entity (XXE) attacks.	1	Link

V2 Validation and Business Logic

(Refer to pages 28–31 in the linked document)

V2.1 Validation and Business Logic Documentation

ID	Description	Level	Reference
2.1.1	Verify that the application documentation defines input validation rules for how to check the validity of data items against an expected structure (e.g., credit card numbers, email addresses).	1	Link

V2.2 Input Validation

ID	Description	Level	Reference
2.2.1	Verify that input is validated to enforce business or functional expectations. This can be positive validation using an allow list or comparing input to predefined rules. For Level 1, focus on business or security decisions.	1	Link
2.2.2	Verify that the application is designed to enforce input validation at a trusted service layer. Client-side validation is useful but cannot be relied upon as a security control.	1	

V2.3 Business Logic Security

ID	Description	Level	Reference
2.3.1	Verify that the application will only process business logic flows for the same user in the expected sequential step order, without skipping steps.	1	Link

V3 Web Frontend Security

(Refer to pages 28–31 in the linked document 32- 37)

V3.2 Unintended Content Interpretation

ID	Description	Level
3.2.1	Verify that security controls prevent browsers from rendering content in the incorrect context (e.g., user-uploaded files, APIs).	1
3.2.2	Verify that content intended as text is rendered using safe methods such as createTextNode, text Content, or equivalents.	1

V3.3 – Cookie Setup

ID	Description	Level	Reference
3.3.1	Verify that cookies use the Secure attribute and appropriate prefixes like __Secure- and __Host-.	1	Link

V3.4 – Browser Security Mechanism Headers

ID	Description	Level
3.4.1	Verify that the Strict-Transport-Security (HSTS) header is present with a max-age of at least 1 year and includes subdomains.	1
3.4.2	Verify that Access-Control-Allow-Origin is set to a specific origin or a validated origin and does not expose sensitive data when * is used.	1

V3.5 – Browser Origin Separation

ID	Description	Level	Reference
3.5.1	Verify that sensitive requests use anti-CSRF mechanisms like tokens or custom headers when preflight is not used.	1	Link
3.5.2	Verify that CORS preflight protections cannot be bypassed with simple requests or headers.	1	
3.5.3	Ensure sensitive operations use appropriate HTTP methods and/or validate Sec-Fetch-* headers.	1	

V4: API and Web Service Security

(Refer to pages 38–41 in the linked document)

V4.1 – Generic Web Service Security

ID	Description	Level	Reference
4.1.1	Verify that every HTTP response with a message body contains a Content-Type header field that matches the actual content of the response, including the charset parameter (e.g., UTF-8, ISO-8859-1).	1	Link

V4.4 – WebSocket Security

ID	Description	Level	Reference
4.4.1	Verify that WebSocket over TLS (WSS) is used for all WebSocket connections.	1	Link

V5 File Handling

(Refer to pages 41–43 in the linked document)

V5.2 File Upload and Content

ID	Description	Level	Reference
5.2.1	Verify that the application will only accept files of a size which it can process without causing a loss of performance or a denial-of-service attack.	1	Link
5.2.2	Verify that when the application accepts a file, either on its own or within an archive (such as a zip file), it checks if the file extension matches an expected file extension and validates that the contents correspond to the type represented by the extension. This includes, but is not limited to, checking the initial 'magic bytes', performing image re-writing, and using specialized libraries for file content validation. For Level 1, this can focus just on files used to make specific business or security decisions.	1	

V5.3 File Storage

ID	Description	Level	Reference
5.3.1	Verify that files uploaded or generated by untrusted input and stored in a public folder are not executed as server-side program code when accessed directly with an HTTP request.	1	Link

V6 Authentication

(Refer to pages 44–52 in the linked document)

V6.1 Authentication Documentation

ID	Description	Level	Reference
6.1.1	Verify that application documentation defines how controls such as rate limiting, anti-automation, and adaptive response are used to defend against attacks such as credential stuffing and password brute force. The documentation must make clear how these controls are configured and prevent malicious account lockout.	1	Link

V6.2 Password Security

ID	Description	Level	Reference
6.2.1	Verify that user set passwords are at least 8 characters in length, although a minimum of 15 characters is strongly recommended.	1	Link
6.2.2	Verify that users can change their password.	1	
6.2.3	Verify that password change functionality requires the user's current and new password.	1	
6.2.4	Verify that passwords submitted during account registration or password change are checked against an available set of, at least, the top 3000 passwords which match the application's password policy (e.g., minimum length).	1	
6.2.5	Verify that passwords of any composition can be used, without rules limiting the type of characters permitted. There must be no requirement for a minimum number of uppercase or lowercase characters, numbers, or special characters.	1	
6.2.6	Verify that password input fields use type="password" to mask the entry. Applications may allow the user to temporarily view the entire masked password or the last typed character of the password.	1	
6.2.7	Verify that "paste" functionality, browser password helpers, and external password managers are permitted.	1	
6.2.8	Verify that the application verifies the user's password exactly as received from the user, without any modifications such as truncation or case transformation.	1	

V6.3 General Authentication Security

ID	Description	Level	Reference
6.3.1	Verify that controls to prevent attacks such as credential stuffing and password brute force are implemented according to the application's security documentation.	1	Link
6.3.2	Verify that default user accounts (e.g., "root", "admin", or "sa") are not present in the application or are disabled.	1	

V6.4 Authentication Factor Lifecycle and Recovery

ID	Description	Level	Reference
6.4.1	Verify that system generated initial passwords or activation codes are securely randomly generated, follow the existing password policy, and expire after a short period of time or after they are initially used. These initial secrets must not be permitted to become the long-term password.	1	Link
6.4.2	Verify that password hints or knowledge-based authentication (so-called “secret questions”) are not present.	1	

V7 Session Management

(Refer to pages 52–56 in the linked document)

V7.2 Fundamental Session Management Security

ID	Description	Level	Reference
7.2.1	Verify that the application performs all session token verification using a trusted, backend service.	1	Link
7.2.2	Verify that the application uses either self-contained or reference tokens that are dynamically generated for session management, i.e., not using static API secrets and keys.	1	
7.2.3	Verify that if reference tokens are used to represent user sessions, they are unique and generated using a cryptographically secure pseudo-random number generator (CSPRNG) and possess at least 128 bits of entropy.	1	
7.2.4	Verify that the application generates a new session token on user authentication, including re-authentication, and terminates the current session token.	1	

V7.4 Session Termination

ID	Description	Level
7.4.1	Verify that when session termination is triggered (such as logout or expiration), the application disallows any further use of the session. For reference tokens or stateful sessions, this means invalidating the session data at the application backend. Applications using self-contained tokens will need solutions such	1

	as maintaining a list of terminated tokens, disallowing tokens produced before a per-user date and time, or rotating a per-user signing key.	
7.4.2	Verify that the application terminates all active sessions when a user account is disabled or deleted (such as an employee leaving the company).	1

V8 Authorization

(Refer to pages 56–59 in the linked document)

V8.1 Authorization Documentation

ID	Description	Level	Reference
8.1.1	Verify that authorization documentation defines rules for restricting function-level and data-specific access based on consumer permissions and resource attributes.	1	Link

V8.2 General Authorization Design

ID	Description	Level	Reference
8.2.1	Verify that the application ensures that functional access is restricted to consumers with explicit permissions.	1	Link
8.2.2	Verify that the application ensures that data-specific access is restricted to consumers with explicit permissions to specific data items to mitigate insecure direct object reference (IDOR) and broken object level authorization (BOLA).	1	

V8.3 Operation Level Authorization

ID	Description	Level	Reference
8.3.1	Verify that the application enforces authorization rules at a trusted service layer and doesn't rely on controls that an untrusted consumer could manipulate, such as client-side JavaScript.	1	Link

V9 Self-contained Tokens

(Refer to pages 59–61 in the linked document)

V9.1 Token Source and Integrity

ID	Description	Level	Reference
9.1.1	Verify that self-contained tokens are validated using their digital signature or MAC to protect against tampering before accepting the token's contents.	1	Link
9.1.2	Verify that only algorithms on an allowlist can be used to create and verify self-contained tokens, for a given context. The allowlist must include the permitted algorithms, ideally only either symmetric or asymmetric algorithms, and must not include the 'None' algorithm. If both symmetric and asymmetric must be supported, additional controls will be needed to prevent key confusion.	1	
9.1.3	Verify that key material that is used to validate self-contained tokens is from trusted pre-configured sources for the token issuer, preventing attackers from specifying untrusted sources and keys. For JWTs and other JWS structures, headers such as 'jku', 'x5u', and 'jwk' must be validated against an allowlist of trusted sources.	1	

V9.2 Token Content

ID	Description	Level	Reference
9.2.1	Verify that, if a validity time span is present in the token data, the token and its content are accepted only if the verification time is within this validity time span. For example, for JWTs, the claims 'nbf' and 'exp' must be verified.	1	Link

V10 OAuth and OIDC

(Refer to pages 61–69 in the linked document)

V10.4 OAuth Authorization Server

ID	Description	Level	Reference
10.4.1	Verify that the authorization server validates redirect URIs based on a client-specific allowlist of pre-registered URIs using exact string comparison.	1	Link
10.4.2	Verify that, if the authorization server returns the authorization code in the authorization response, it can be used only once for a token request. For the second valid request with an authorization code that has already been used to issue an access token, the authorization server must reject a token request and revoke any issued tokens related to the authorization code.	1	
10.4.3	Verify that the authorization code is short-lived. The maximum lifetime can be up to 10 minutes for L1 and L2 applications and up to 1 minute for L3 applications.	1	
10.4.4	Verify that for a given client, the authorization server only allows the usage of grants that this client needs to use. Note that the grants 'token' (Implicit flow) and 'password' (Resource Owner Password Credentials flow) must no longer be used.	1	

V11 Cryptography

(Refer to pages 70–75 in the linked document)

V11.3 Encryption Algorithms

ID	Description	Level	Reference
11.3.1	Verify that insecure block modes (e.g., ECB) and weak padding schemes (e.g., PKCS#1 v1.5) are not used.	1	Link
11.3.2	Verify that only approved ciphers and modes such as AES with GCM are used.	1	

V11.4 Hashing and Hash-based Functions

ID	Description	Level	Reference
11.4.1	Verify that only approved hash functions are used for general cryptographic use cases, including digital signatures, HMAC, KDF, and random bit generation. Disallowed hash functions, such as MD5, must not be used for any cryptographic purpose.	1	Link

V12 Secure Communication

(Refer to pages 75–77 in the linked document)

V12.1 General TLS Security Guidance

ID	Description	Level	Reference
12.1.1	Verify that only the latest recommended versions of the TLS protocol are enabled, such as TLS 1.2 and TLS 1.3. The latest version of the TLS protocol must be the preferred option.	1	Link

V12.2 HTTPS Communication with External Facing Services

ID	Description	Level	Reference
12.2.1	Verify that TLS is used for all connectivity between a client and external facing, HTTP-based services, and does not fall back to insecure or unencrypted communications.	1	Link
12.2.2	Verify that external facing services use publicly trusted TLS certificates.	1	

V13 Configuration

(Refer to pages 77–81 in the linked document)

V13.4 Unintended Information Leakage

ID	Description	Level	Reference
13.4.1	Verify that the application is deployed without any source control metadata, including the .git or .svn folders, or in a way that these folders are inaccessible both externally and to the application itself.	1	Link

V14 Data Protection

(Refer to pages 81 – 84 in the linked document)

V14.2 General Data Protection

ID	Description	Level	Reference
14.2.1	Verify that sensitive data is only sent to the server in the HTTP message body or header fields, and that the URL and query string do not contain sensitive information, such as an API key or session token.	1	Link1 Link2 Link3 Link4

V14.3 Client-Side Data Protection

ID	Description	Level	Reference
14.3.1	Verify that authenticated data is cleared from client storage, such as the browser DOM, after the client or session is terminated. The 'Clear-Site-Data' HTTP response header field may be able to help with this, but the client-side should also be able to clear up if the server connection is not available when the session is terminated.	1	Link1 Link2 Link3

V15 Secure Coding and Architecture

(Refer to pages 84–88 in the linked document)

V15.1 Secure Coding and Architecture Documentation

ID	Description	Level	Reference
15.1.1	Verify that application documentation defines risk-based remediation time frames for 3rd party component versions with vulnerabilities and for updating libraries in general, to minimize the risk from these components.	1	Link

V15.2 Security Architecture and Dependencies

ID	Description	Level	Reference
15.2.1	Verify that the application only contains components which have not breached the documented update and remediation time frames.	1	Link

V15.3 Defensive Coding

ID	Description	Level	Reference
15.3.1	Verify that the application only returns the required subset of fields from a data object. For example, it should not return an entire data object, as some individual fields should not be accessible to users.	1	Link

Note: All the above content is based on the **OWASP Application Security Verification Standard (ASVS)** documentation. You can refer to the official standard here:

🔗 [OWASP ASVS Website](#)

🔗 [OWASP ASVS Document](#)

The **Level 1 (L1)** and **Level 2 (L2)** security testing requirements are applicable to **both Web and Desktop applications**. However, during Desktop application testing, **additional checkpoints** may be included to address platform-specific risks and functionalities

References

1. [OWASP Web Security Testing Guide: Input Validation Testing](#)
2. [OWASP Web Security Testing Guide: Business Logic Testing](#)
3. [OWASP LDAP Injection Prevention Cheat Sheet](#)
4. [OWASP Cross Site Scripting Prevention Cheat Sheet](#)
5. [OWASP DOM Based Cross Site Scripting Prevention Cheat Sheet](#)
6. [OWASP Top 10 Project: https://owasp.org/www-project-top-ten/](https://owasp.org/www-project-top-ten/)

7. OWASP Web Security Testing Guide:
<https://owasp.org/www-project-web-security-testing-guide/>
8. https://github.com/OWASP/ASVS/raw/v5.0.0/5.0/OWASP_Application_Security_Verification_Standard_5.0.0_en.pdf